

Uv



Python Package Management



# The Ultimate Guide to uv

The Last Python Package Manager You'll Ever Need



George Pipis

Follow

12 min read · Apr 13, 2026



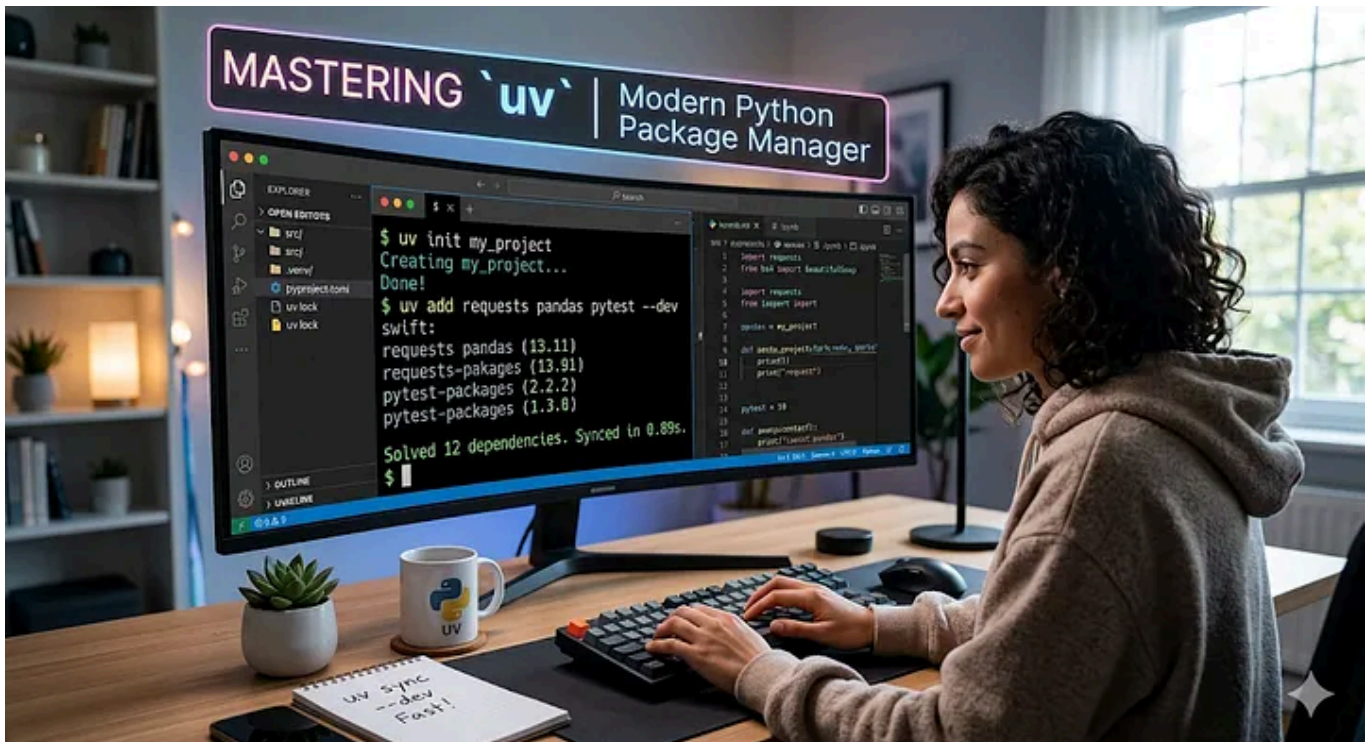
458



6



1



Generated by authoer using Gemini

If you've spent any time in the Python ecosystem, you know the packaging and dependency management story can be... messy. Between `pip`, `venv`,

`pip-tools`, `pyenv`, `poetry`, and `pipx`, setting up a reliable local environment often feels like a delicate juggling act.

Enter `uv`.

Created by Astral (the same team behind the incredibly fast `ruff` linter), `uv` is an extremely fast Python package and project manager written in Rust. It is designed to replace almost all of the tools mentioned above, offering a single, unified, and blisteringly fast workflow.

In this tutorial, we'll walk through `uv` from the ground up, from installation and basic project management to advanced features like script execution and sharing your project with others.

## Step 1: Installing `uv`

Because `uv` manages Python, it's best to install it *outside* of Python. Astral provides standalone installers.

### macOS and Linux:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

### Windows:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | ie
```

*(Note: You can also use Homebrew on macOS with `brew install uv`, or install it via `pip` if you prefer, though the standalone installer is recommended.)*

Once installed, verify it's ready to go:

```
uv --version
```

## Step 2: Managing Python Versions (The `pyenv` Killer)

You no longer need external tools to install different versions of Python. `uv` handles downloading and managing Python toolchains automatically.

To install a specific version of Python:

```
uv python install 3.12
```

To see which versions you have installed:

```
uv python list
```

## Step 3: Initializing a Project

Let's build a project. Unlike the old `mkdir` and `python -m venv venv` workflow, `uv` acts more like Rust's `cargo` or Node's `npm`.

## Create a new directory and initialize a project:

```
uv init my_cool_project  
cd my_cool_project
```

This generates a few files for you, most importantly:

- `pyproject.toml` : The standard configuration file for your project and its dependencies.
- `hello.py` : A basic starter script.
- `.python-version` : A file dictating which Python version this project uses.

### Step 4: Managing Dependencies

Let's say we want to build a simple web scraper. We'll need the `requests` and `beautifulsoup4` libraries.

To add dependencies, use the `uv add` command:

```
uv add requests beautifulsoup4
```

### What just happened behind the scenes?

1. `uv` noticed you didn't have a virtual environment yet, so it created one automatically in a `.venv` folder.

2. It downloaded and installed the packages (in milliseconds, thanks to its global cache).
3. It updated your `pyproject.toml` to include these dependencies.
4. It created a `uv.lock` file to pin the exact versions of all your dependencies and their sub-dependencies.

Need a development-only dependency, like `pytest`?

```
uv add --dev pytest
```

## Step 5: Running Your Code

With traditional tools, you have to manually activate your virtual environment ( `source .venv/bin/activate` ). With `uv`, you can skip that entirely using `uv run`.

`uv run` automatically ensures your environment is up-to-date and executes the command *inside* the virtual environment.

Let's modify the generated `hello.py` file:

```
import requests

response = requests.get("https://api.github.com")
print(f"GitHub API Status: {response.status_code}")
```

Now, run it:

```
uv run hello.py
```

## Step 6: Running Global Tools (The `pipx` Killer)

Sometimes you want to run a Python CLI tool (like `ruff`, `httpie`, or `cowsay`) without polluting your global Python environment or setting up a whole new project.

`uv` includes a command called `uvx` (or `uv tool run`), which downloads and runs a tool in an isolated, ephemeral environment.

```
uvx cowsay "uv is incredibly fast!"
```

If you want to install a tool permanently so it's available in your terminal everywhere:

```
uv tool install ruff
```

## Step 7: Advanced Use — Single-File Scripts

One of `uv`'s most powerful features is its support for PEP 723: inline script metadata. This allows you to define dependencies *inside* a single Python script, without needing a `pyproject.toml` or a directory structure.

Create a file called `script.py`:

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "rich",
# ]
# ///

from rich import print

print("[bold green>Hello from an isolated script![/bold green]")
```

Run it with `uv run script.py`. `uv` will automatically read the comment block, create an ephemeral environment with `rich` installed, run the script, and clean up. This is phenomenal for sharing quick automation scripts with coworkers.

## Step 8: Sharing Your Project

When it's time to share your project with teammates or deploy it to a server, `uv` makes it incredibly simple and highly reproducible.

### 1. Version Control

You should commit the following files to your Git repository:

- `pyproject.toml`
- `uv.lock`
- `.python-version`
- Your source code

**Do not** commit the `.venv` directory.

## 2. Onboarding a Teammate

When a coworker clones your repository, they only need `uv` installed. They don't even need to worry about installing the right version of Python manually.

All they have to run is:

```
uv sync
```

This command will:

1. Read the `.python-version` file and download that exact Python version if they don't have it.
2. Create a fresh `.venv`.
3. Read the `uv.lock` file and install the exact, mathematically reproducible dependency tree.

## 3. Building and Publishing

If you are building a library meant to be shared on PyPI, `uv` can handle the build and publish steps as well.

To build your wheels and source distributions:

```
uv build
```

To publish to PyPI:

```
uv publish
```

## Differences Between `uv add`, `uv tool install` and `uv pip install`

While `uv add`, `uv tool install`, and `uv pip install` all download and install Python packages, they serve entirely different purposes in your workflow.

Think of them like this: `uv add` is for **managing a specific project**, `uv tool install` is for **installing global apps**, and `uv pip install` is for **low-level, manual environment tweaking**.

Here is the detailed breakdown of how and when to use each one.

### 1. `uv add` (The Project Manager)

This command is used when you are developing a specific project (like a web app or a library) and you want to declare a dependency that your code relies on.

- **What it does:** It adds the package to your `pyproject.toml` file, calculates the exact required versions and writes them to a `uv.lock` file, and installs the package into your project's local virtual environment (usually a `.venv` folder).

- **The Analogy:** It is `uv` 's equivalent to `poetry add`, `npm install <pkg>`, or `cargo add`.
- **When to use it:** When you are writing code and need to import a library.  
*Example:* `uv add requests` (because your script needs to make HTTP requests).

## 2. `uv tool install` (The Global App Installer)

This command is used to install Python-based command-line interface (CLI) tools that you want to use globally in your terminal, without polluting your current project's environment.

- **What it does:** It creates a dedicated, isolated, hidden virtual environment *just* for that specific tool, and then links the executable to your system's PATH. It does not touch your `pyproject.toml` or local `.venv`.
- **The Analogy:** It is `uv` 's equivalent to `pipx install` or `brew install`.
- **When to use it:** When you want an executable tool available anywhere on your computer, regardless of what folder you are in.
- *Example:* `uv tool install ruff` (so you can run the `ruff` linter in any directory) or `uv tool install httpie`.

## 3. `uv pip install` (The Low-Level Interface)

This command is a direct, drop-in replacement for the standard Python `pip install` command, just written in Rust so it executes much faster.

- **What it does:** It blindly installs a package into whatever Python environment is currently active. It **does not** update your `pyproject.toml`

and it **does not** create a lockfile.

- **The Analogy:** It is exactly `pip install`, just faster.
- **When to use it:** When you are dealing with legacy workflows, working inside a Docker container where you don't need lockfiles, or you just want to quickly test a library in an active environment without making it a permanent part of your project.
- *Example:* `uv pip install django` (just to play around with it temporarily in an active virtual environment).

| Feature                   | uv add                                    | uv tool install                  | uv pip install                   |
|---------------------------|-------------------------------------------|----------------------------------|----------------------------------|
| Updates pyproject.toml?   | Yes                                       | No                               | No                               |
| Creates/Updates Lockfile? | Yes                                       | No                               | No                               |
| Where does it install?    | The project's .venv                       | A hidden, isolated global folder | The currently active environment |
| Primary Use Case          | Building projects & managing dependencies | Installing global CLI tools      | Manual environment modification  |
| Replaces...               | poetry add, pipenv install                | pipx install                     | pip install                      |

### What about Jupyter Notebooks?

Integrating Jupyter Notebooks into a project can historically be a bit messy. The common mistake is installing `jupyter` or `jupyterlab` directly into your main dependencies, which severely bloats your production environment.

With `uv`, you have a few incredibly elegant ways to handle Jupyter, depending on your workflow. Here are the best practices, ranked from the "cleanest" modern approach to the traditional team approach.

### Approach 1: The “Zero-Bloat” Ephemeral Run (Recommended for Solo/Ad-Hoc)

If you just want to spin up a notebook to explore the data in your project, you **don't** need to install Jupyter into your `pyproject.toml` at all.

You can use `uv run` with the `--with` flag. This tells `uv` to temporarily download Jupyter, combine it with your project's existing virtual environment, and run it.

```
# From inside your project directory:  
uv run --with jupyter jupyter lab
```

### Why this is great:

- **Zero Bloat:** Your `pyproject.toml` and `uv.lock` stay pristine. No Jupyter dependencies cluttering up your actual application logic.
- **Full Access:** Because you used `uv run`, the notebook still has complete access to all the packages installed in your project's `.venv` (like `pandas` or `requests`).
- **Always Up-to-Date:** It will grab the latest compatible version of Jupyter on the fly (which takes milliseconds after the first cached run).

### Approach 2: The IDE Approach (VS Code / PyCharm)

If you write your notebooks directly inside VS Code or PyCharm, you don't even need to run a Jupyter server yourself. The IDE handles the server; it just needs a “kernel” (an execution environment) to run the code.

1. Add `ipykernel` to your development dependencies. This allows your IDE to talk to your `uv` environment.

```
uv add --dev ipykernel
```

2. Open your `.ipynb` file in VS Code.
3. Click the “**Select Kernel**” button in the top right.
4. Choose “**Python Environments**” and point it to the `.venv` folder inside your `uv` project directory.

**Why this is great:** It’s the fastest workflow if you rely on an editor, and it correctly sandboxes the required kernel package as a dev-only dependency.

### Approach 3: The Traditional “Dev Dependency” (Recommended for Teams)

If you are working on a data science team where *everyone* is expected to spin up a Jupyter server to work on the repository, you should formalize it in your project configuration. However, you should strictly keep it as a **development dependency**.

1. Add Jupyter to your dev group:

```
uv add --dev jupyterlab
```

2. Now, anyone who clones the repo and runs `uv sync` will get Jupyter installed in their local `.venv`.

3. To start the notebook server, they simply run:

```
uv run jupyter lab
```

**Why this is great:** It ensures reproducibility. Every developer on your team will be using the exact same version of Jupyter Lab, locked by your `uv.lock` file, without it leaking into your production build.

#### Approach 4: The Global Tool + Custom Kernel (For Heavy Jupyter Users)

If you manage dozens of projects and don't want to install or run Jupyter separately for each one, you can install Jupyter globally as a `uv tool`, and then "register" your project environments to that global server.

1. Install Jupyter globally (you only do this once on your machine):

```
uv tool install jupyter
```

2. Inside your specific project, add the kernel as a dev dependency:

```
uv add --dev ipykernel
```

3. Register this project's environment as a kernel available to your global Jupyter:

```
uv run python -m ipykernel install --user --name="my_cool_project" --display-name="My Cool Project (uv)"
```

4. Start your global Jupyter from anywhere:

```
uvx jupyter lab
```

(When the UI opens, you will see “My Cool Project (uv)” as an option in the launcher).

**Summary: Which should you choose?**

- Use **Approach 1** (`uv run --with jupyter`) if you just need to quickly visualize something and don't want to change any project files.
- Use **Approach 2** (VS Code + `ipykernel`) if you live inside your IDE.
- Use **Approach 3** (`uv add --dev jupyterlab`) if you are building a shared data science repo with teammates.

**Further explanation of the `--dev` concept**

The `--dev` (development dependency) concept is one of those things that trips up almost every developer early on because, when you are just writing code on your own laptop, everything feels exactly the same.

To understand `--dev`, it helps to think about **building a house**.

When you build a house, there are two distinct categories of things you need:

- 1. The Materials (Production Dependencies):** These are the bricks, the plumbing, the wood, and the glass. The house *literally cannot function* without them. If you take the pipes away, the house breaks. *In Python:* These are packages like `requests` (to fetch data), `FastAPI` (to run a web server), or `pandas` (to process data). Your app will crash if these aren't installed. You install these with `uv add <package>`.
- 2. The Tools (Development Dependencies):** These are the cranes, the scaffolding, the blueprints, and the hardhats. The construction workers *need* these to build the house. But once the house is finished and someone moves in, you take all of those tools away. You don't leave a massive crane sitting in the living room. *In Python:* These are packages like `pytest` (to test your code), `ruff` (to format your code), or `jupyterlab` (to experiment). Your actual app doesn't need these to run; *you* need them to write the app. You install these with `uv add --dev <package>`.

**Why do we bother separating them?**

When you are working on your laptop, `uv` installs *both* the materials and the tools so you can get to work. But the separation becomes incredibly important when you want to share or deploy your code:

- **Deployment (Moving into the house):** When you put your code on a server (like AWS) or package it into a Docker container, you tell `uv` to **only** install the production dependencies (e.g., using a command like `uv sync --no-dev`). This keeps your server lightweight, fast, and secure. You aren't wasting server space downloading testing tools that the server will never use.
- **Collaboration (Hiring more builders):** When a teammate downloads your project, they need the tools just like you do. They run a standard `uv sync`, and `uv` knows to give them the production packages *plus* all the dev packages, ensuring their testing environment perfectly matches yours.

## What it looks like under the hood

When you use the `--dev` flag, `uv` updates your `pyproject.toml` file to keep the lists separate. It usually looks something like this:

```
[project]
name = "my_app"
# The Bricks (Production)
dependencies = [
    "requests",
]

[tool.uv]
# The Scaffolding (Development)
dev-dependencies = [
    "pytest",
    "jupyterlab",
]
```

By keeping them separated, you ensure your production environment stays clean while your local development environment has all the bells and

whistles you need to be productive.

Are you planning to deploy your project to a live server soon, or are you mostly building this for local use right now?

## The Magic of `uv sync`

Once you have your project set up and your dependencies locked in `uv.lock`, you need a way to build your virtual environment. `uv sync` is the command that reads your lockfile and makes sure your local `.venv` perfectly matches it.

However, *how* you sync changes entirely depending on where the code is running:

### `uv sync` (The Default)

- **What it does:** Installs both your production “materials” AND your development “tools” (like `pytest` or `jupyterlab`).
- **When to use it:** When a teammate clones your repository to their local laptop. They need everything you had so they can run tests, format code, and contribute to the project.

### `uv sync --no-dev` (The Deployment Hero)

- **What it does:** Strips out all the development tools and strictly installs your core production dependencies.
- **When to use it:** When you are writing a `Dockerfile`, deploying to a cloud server, or pushing a release to production. Your server doesn't need

testing frameworks or Jupyter notebooks; it just needs the core application to run as fast and securely as possible.

By taking the time to separate your `--dev` tools from the start, a single `--no-dev` flag is all it takes to instantly transition your project from a cluttered local workbench to a clean, production-ready application!

## Bonus: Quality of Life Features

### 1. Automatic `.env` File Loading

In standard Python, if you have API keys or database passwords in a `.env` file, you usually have to install a library like `python-dotenv` and add boilerplate code to load it.

`uv` handles this out of the box. If you have a `.env` file in your project directory, `uv run` automatically loads those environment variables before executing your script.

**`.env` file:**

```
SECRET_API_KEY=super_secret_123
```

**`main.py`:**

```
import os
print(os.environ.get("SECRET_API_KEY"))
```

Just run `uv run main.py`, and it works instantly. No extra libraries required!

## 2. Painless Migration from `pip`

Your readers might be wondering, *“This sounds great for a new project, but what about my existing projects?”*

`uv` makes migrating incredibly easy. If you already have a `requirements.txt` file from an old project, you can initialize `uv` and import your old dependencies in one fell swoop:

```
# Initialize uv in your existing folder
uv init

# Import your existing requirements.txt
uv add -r requirements.txt
```

`uv` will parse your old requirements, find the mathematically optimal dependency tree, and generate your modern `pyproject.toml` and `uv.lock` files in seconds. You can then delete your old `requirements.txt` forever!

## Conclusion: The New Standard for Python

If you look back at everything we’ve covered, it’s easy to see why `uv` is taking the Python community by storm. In a single, incredibly fast Rust binary, we

[Open in app](#) ↗



Medium



Search



Write



- `venv` is handled automatically in the background

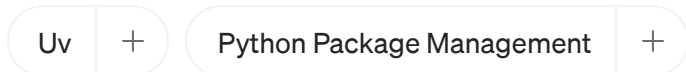
- **pip** and **pip-tools** are replaced by `uv add` and `uv.lock`
- **pipx** is replaced by `uvx` and `uv tool install`
- **poetry** and **pipenv** are replaced by `uv` 's native project management

The days of juggling half a dozen different CLIs, writing complex `.env` loading boilerplate, or accidentally leaking Jupyter notebooks into your production servers are over.

By adopting `uv` , you get blazing-fast dependency resolution, mathematically reproducible environments, a crystal-clear separation between local development tools and production builds, and a frictionless experience for everything from single-file automation scripts to massive web applications.

The Python packaging ecosystem used to be a punchline. With `uv` , it's finally a superpower.

*Happy coding!*



**Written by George Pipis**

1.2K followers · 389 following

Sr Manager @Accenture

Follow



## Responses (6)



Alex Mylnikov

What are your thoughts?



silvasta

Apr 27



The Jupyter connection was actually quite annoying... until I figured out this shell script:  
(the export is just for having the only variable at the top instead of figuring out again and again somewhere)

# connect to Jupyter

export... [more](#)



4

[Reply](#)



Nikolai von Krusenstiern

May 16



I am currently full in with Astral Python tooling .. ruff , uv , ty .

But we had bad experience with big tech, when it comes to open source. And OpenAI buying Astral destroys my hope for a modern non fragmented Python stack ... :-(



3

[Reply](#)



Alexandre Couëdelo

5 hours ago



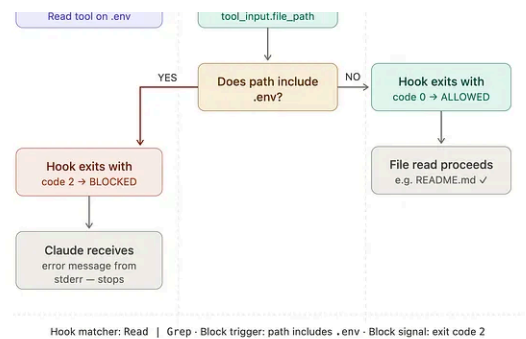
uv is getting a lot of things right, such a shame that the python ecosystem is so fragmented



[Reply](#)

See all responses

More from George Pipis

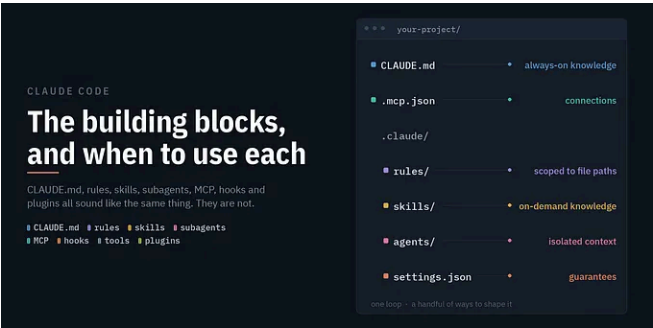


 George Pipis · May 21

**Protecting Your .env File with Claude Code Hooks**  
A Step-by-Step Tutorial for Beginners

 3  1

 ...

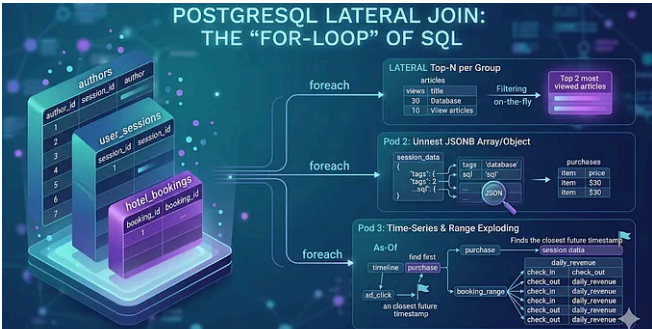


 George Pipis · Jul 24

**CLAUDE.md, Skills, Rules, Hooks, MCP, Agents**  
What each one is and when to use it

 101

 ...



 George Pipis · Apr 14

## Demystifying PostgreSQL LATERAL Joins

The “For-Loop” of SQL

 7  1

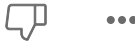


 George Pipis · Nov 30, 2024

## Mastering Euroleague Fantasy with Data-Driven Decisions

Leverage web scraping, advanced metrics, and Monte Carlo simulations to dominate...

 228



See all from George Pipis

